

---

# TP : Détection de Piétons

avec YOLO et Validation par Opérateur de Sobel

---

Maryam Boufous  
Traitement d'image  
Ibn Zohr, Ensa Agadir

|                                                                                         |
|-----------------------------------------------------------------------------------------|
| <b>Module :</b> Traitement d'images <b>Prérequis :</b> C++, traitement d'images, OpenCV |
|-----------------------------------------------------------------------------------------|

## Abstract

Ce travail pratique propose d'implémenter un système complet de détection de piétons alliant le deep learning (YOLOv4-tiny) et les méthodes classiques de vision par ordinateur (opérateur de Sobel). La combinaison des deux approches améliore la robustesse et permet de valider les détections par l'analyse des structures verticales caractéristiques des piétons.

## Contents

---

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction à la détection de piétons</b>                  | <b>3</b>  |
| 1.1      | Contexte et enjeux . . . . .                                   | 3         |
| 1.2      | Approches de détection . . . . .                               | 3         |
| <b>2</b> | <b>Concepts fondamentaux</b>                                   | <b>3</b>  |
| 2.1      | YOLO — You Only Look Once . . . . .                            | 4         |
| 2.2      | NMS — Non-Maximum Suppression . . . . .                        | 5         |
| 2.3      | Opérateur de Sobel . . . . .                                   | 6         |
| 2.4      | Approche hybride : pourquoi combiner YOLO et Sobel ? . . . . . | 7         |
| <b>3</b> | <b>Outils et bibliothèques</b>                                 | <b>8</b>  |
| 3.1      | OpenCV . . . . .                                               | 8         |
| <b>4</b> | <b>Architecture du pipeline</b>                                | <b>8</b>  |
| <b>5</b> | <b>Étapes détaillées du pipeline</b>                           | <b>9</b>  |
| 5.1      | Étape 1 : Configuration et chargement de l'image . . . . .     | 9         |
| 5.2      | Étape 2 : Chargement du modèle YOLO . . . . .                  | 9         |
| 5.3      | Étape 3 : Création du blob d'entrée . . . . .                  | 10        |
| 5.4      | Étape 4 : Inférence YOLO (forward pass) . . . . .              | 10        |
| 5.5      | Étape 5 : Extraction des détections brutes . . . . .           | 10        |
| 5.6      | Étape 6 : Non-Maximum Suppression (NMS) . . . . .              | 11        |
| 5.7      | Étape 7 : Validation Sobel (3 méthodes + vote) . . . . .       | 11        |
| 5.8      | Étape 8 : Visualisation et codage couleur . . . . .            | 14        |
| 5.9      | Étape 9 : Sauvegarde et affichage final . . . . .              | 15        |
| <b>6</b> | <b>Travail pratique à réaliser</b>                             | <b>16</b> |
| 6.1      | Partie 1 : Prise en main . . . . .                             | 16        |
| 6.2      | Partie 2 : Analyse des paramètres . . . . .                    | 16        |
| <b>7</b> | <b>Questions théoriques</b>                                    | <b>17</b> |
| <b>8</b> | <b>Annexes</b>                                                 | <b>18</b> |
| 8.1      | Installation et compilation . . . . .                          | 18        |
| 8.2      | Rappel — Opérateurs de convolution . . . . .                   | 18        |
| 8.3      | Paramètres recommandés . . . . .                               | 19        |
| 8.4      | Dépannage . . . . .                                            | 19        |
| 8.5      | Références . . . . .                                           | 19        |

## List of Figures

---

|   |                                                                                                                     |   |
|---|---------------------------------------------------------------------------------------------------------------------|---|
| 1 | Architecture de YOLOv4-tiny : du blob d'entrée aux prédictions finales . .                                          | 4 |
| 2 | Format des sorties YOLO et conversion vers les coordonnées pixel . . . .                                            | 4 |
| 3 | Avant/après NMS : suppression des boîtes redondantes sur le même piéton                                             | 5 |
| 4 | Décomposition Sobel sur une silhouette : image originale, $G_x$ , $G_y$ , magni-<br>tude $\ \mathbf{G}\ $ . . . . . | 7 |
| 5 | Vue d'ensemble du pipeline hybride complet . . . . .                                                                | 7 |

---

|   |                                                                                                                          |    |
|---|--------------------------------------------------------------------------------------------------------------------------|----|
| 6 | Architecture complète du pipeline de détection de piétons hybride . . . . .                                              | 9  |
| 7 | Les 3 méthodes de validation Sobel : orientations (gauche), ratio H/V<br>(centre), écart-type texture (droite) . . . . . | 12 |
| 8 | Système de vote majoritaire combinant les 3 méthodes Sobel . . . . .                                                     | 14 |
| 9 | Image de debug Sobel : bleu = gradient horizontal ( $G_x$ ), rouge = gradient<br>vertical ( $G_y$ ) . . . . .            | 15 |

## List of Tables

---

|   |                                                             |    |
|---|-------------------------------------------------------------|----|
| 1 | Comparaison des approches de détection de piétons . . . . . | 3  |
| 2 | Bénéfices de l'approche hybride YOLO + Sobel . . . . .      | 7  |
| 3 | Fonctions OpenCV utilisées dans ce TP . . . . .             | 8  |
| 4 | Récapitulatif des méthodes de validation Sobel . . . . .    | 14 |
| 5 | Opérateurs de convolution courants . . . . .                | 18 |
| 6 | Paramètres ajustables du système . . . . .                  | 19 |

## 1. Introduction à la détection de piétons

### 1.1 Contexte et enjeux

La détection de piétons est l'un des problèmes fondamentaux en vision par ordinateur, particulièrement critique pour les véhicules autonomes et les systèmes d'aide à la conduite (ADAS). Selon l'Organisation Mondiale de la Santé, plus de 270 000 piétons perdent la vie chaque année dans des accidents de la route. Les systèmes de détection automatique peuvent contribuer significativement à réduire ce chiffre en déclenchant des alertes ou des freinages d'urgence.

#### Pourquoi la détection de piétons est-elle difficile ?

- **Variabilité d'apparence** : vêtements, postures, orientations
- **Environnement complexe** : arrière-plans encombrés, occultations partielles
- **Conditions d'éclairage** : ombres, contre-jour, scènes nocturnes
- **Échelle variable** : piétons proches (grande boîte) ou lointains (quelques pixels)
- **Temps réel** : contraintes de calcul pour les systèmes embarqués

### 1.2 Approches de détection

| Approche                                | Avantages                                           | Inconvénients                                        |
|-----------------------------------------|-----------------------------------------------------|------------------------------------------------------|
| Méthodes classiques (HOG, Haar, Sobel)  | Interprétables, rapides, peu de données nécessaires | Moins précises, sensibles aux variations d'apparence |
| Deep Learning (YOLO, SSD, Faster R-CNN) | Très précises, robustes aux variations              | Besoin de grandes données, opacité, plus lent        |
| Approche hybride (ce TP)                | Robustesse accrue, validation croisée               | Complexité d'intégration                             |

Table 1: Comparaison des approches de détection de piétons

## 2. Concepts fondamentaux

## 2.1 YOLO — You Only Look Once

### Définition – YOLO (You Only Look Once)

Algorithme de détection d'objets en temps réel qui traite l'image en **une seule passe** à travers le réseau (d'où son nom). Contrairement aux approches en deux étapes (région proposals + classification), YOLO divise l'image en une grille et prédit simultanément les boîtes englobantes et les classes pour toutes les cellules. **YOLOv4-tiny** est une version allégée optimisée pour les systèmes embarqués :  $\approx 23$  Mo,  $> 30$  FPS sur CPU, 80 classes (COCO dataset).

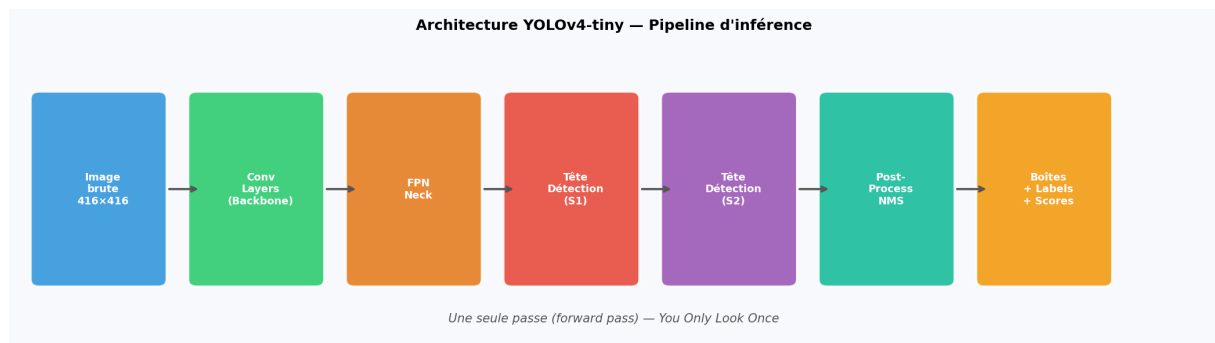


Figure 1: Architecture de YOLOv4-tiny : du blob d'entrée aux prédictions finales

### Format de sortie YOLO

Pour chaque cellule de la grille et chaque ancre (*anchor*), YOLO produit un vecteur de dimension  $5 + C$  (avec  $C = 80$  classes COCO) :

$$\underbrace{[c_x, c_y, w, h]}_{\text{boîte englobante}} \parallel \underbrace{[p_{\text{obj}}]}_{\text{confiance objet}} \parallel \underbrace{[p_0, p_1, \dots, p_{79}]}_{\text{scores de classes}}$$

La classe 0 correspond à *person* (piéton) dans COCO. Les coordonnées sont normalisées entre 0 et 1 et doivent être multipliées par les dimensions de l'image.

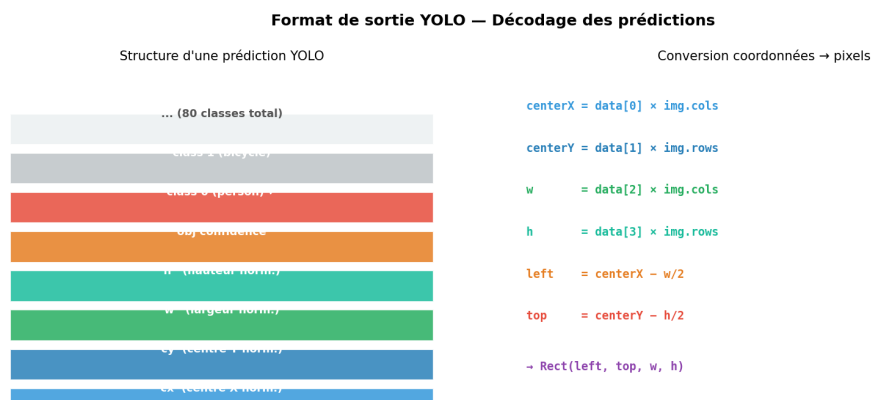


Figure 2: Format des sorties YOLO et conversion vers les coordonnées pixel

## Blob d'entrée

YOLO n'accepte pas directement une image OpenCV. Elle doit être convertie en un **blob** 4D via `blobFromImage()` :

| Dimension     | Valeur | Description         |
|---------------|--------|---------------------|
| batch         | 1      | une seule image     |
| canaux        | 3      | RGB (swapRB = true) |
| hauteur       | 416    | redimensionné       |
| largeur       | 416    | redimensionné       |
| normalisation | ÷255   | valeurs dans [0, 1] |

## 2.2 NMS — Non-Maximum Suppression

### Définition – NMS (Non-Maximum Suppression)

Algorithme de post-traitement qui élimine les détections redondantes. Lorsque plusieurs boîtes englobantes se chevauchent fortement et détectent le même objet, NMS ne conserve que celle avec le score de confiance le plus élevé, en supprimant toutes celles dont le chevauchement (mesuré par l'**IoU**) dépasse un seuil.

**Intersection over Union (IoU) :**

$$\text{IoU}(A, B) = \frac{\text{Aire}(A \cap B)}{\text{Aire}(A \cup B)} \in [0, 1]$$

Un IoU proche de 1 signifie que deux boîtes se superposent presque totalement. Dans ce code, le seuil NMS est fixé à **0.40** : toute boîte qui recouvre à plus de 40% une boîte de confiance supérieure est supprimée.

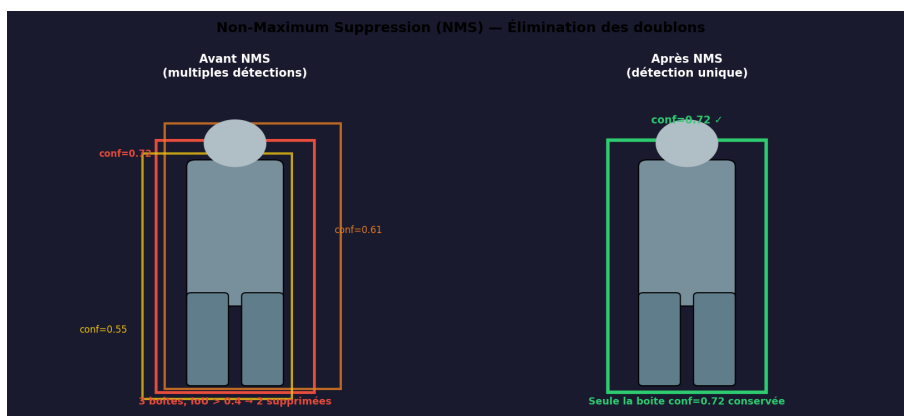


Figure 3: Avant/après NMS : suppression des boîtes redondantes sur le même piéton

## 2.3 Opérateur de Sobel

### Définition – Opérateur de Sobel (1968)

Filtre de détection de contours qui calcule une **approximation du gradient** de l'intensité lumineuse en chaque pixel, en convoluant l'image avec deux noyaux  $3 \times 3$  : l'un sensible aux variations horizontales ( $G_x$ ), l'autre aux variations verticales ( $G_y$ ).

### Noyaux de convolution

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * I \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * I$$

$G_x$  détecte les **contours verticaux** (fort changement selon  $x$ ).  $G_y$  détecte les **contours horizontaux** (fort changement selon  $y$ ).

### Magnitude et orientation du gradient

$$\|\mathbf{G}\| = \sqrt{G_x^2 + G_y^2} \quad \theta = \arctan\left(\frac{G_y}{G_x}\right)$$

### Relation contour / gradient : point essentiel

Le gradient est **perpendiculaire** au contour :

- Un contour **vertical** (bord gauche/droit du corps d'un piéton) produit un fort gradient **horizontal** ( $G_x$  dominant,  $\theta \approx 0^\circ$  ou  $180^\circ$ ).
- Un contour **horizontal** (épaules, ceinture) produit un fort gradient **vertical** ( $G_y$  dominant,  $\theta \approx 90^\circ$ ).

C'est pourquoi on cherche  $\theta < 20^\circ$  ou  $\theta > 160^\circ$  pour détecter les structures **verticales** des piétons.

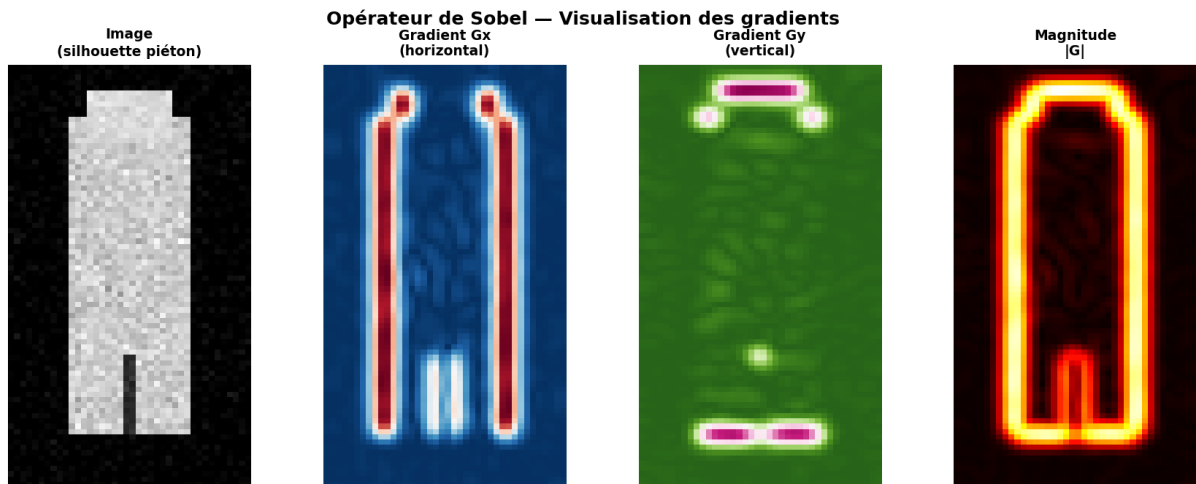


Figure 4: Décomposition Sobel sur une silhouette : image originale,  $G_x$ ,  $G_y$ , magnitude  $\|G\|$

### Dérivée seconde : Sobel d'ordre 2

Dans `validateWithVerticalTexture()`, on utilise la **dérivée seconde verticale** (paramètre `yorder=2` dans `Sobel()`) :

$$G_{yy} = \frac{\partial^2 I}{\partial y^2}$$

Elle est sensible aux variations périodiques de luminosité en vertical, typiques de l'alternance jambes/fond ou bras/corps d'un piéton.

### 2.4 Approche hybride : pourquoi combiner YOLO et Sobel ?

| Critère          | YOLO seul                                 | YOLO + Sobel                      |
|------------------|-------------------------------------------|-----------------------------------|
| Faux positifs    | Possible (occultation, texture similaire) | Réduits par validation classique  |
| Interprétabilité | Faible (boîte noire)                      | Élevée (gradients interprétables) |
| Robustesse       | Bonne en général                          | Meilleure sur cas limites         |
| Vitesse          | Rapide                                    | Légèrement réduite                |

Table 2: Bénéfices de l'approche hybride YOLO + Sobel

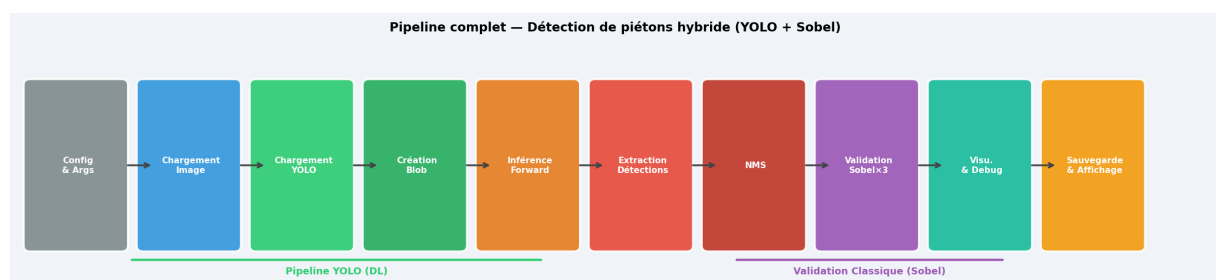


Figure 5: Vue d'ensemble du pipeline hybride complet



### 3. Outils et bibliothèques

#### 3.1 OpenCV

##### Définition – OpenCV (Open Source Computer Vision Library)

Bibliothèque open-source de vision par ordinateur contenant plus de 2500 algorithmes optimisés. Créée par Intel en 1999, elle est le standard de facto en vision par ordinateur, disponible en C++, Python et Java.

| Fonction                     | Module  | Utilisation                                  |
|------------------------------|---------|----------------------------------------------|
| <code>imread()</code>        | core    | Chargement d'image depuis le disque          |
| <code>cvtColor()</code>      | imgproc | Conversion d'espace couleur (BGR, HSV, GRAY) |
| <code>GaussianBlur()</code>  | imgproc | Réduction du bruit avant Sobel               |
| <code>Sobel()</code>         | imgproc | Calcul des gradients (ordre 1 ou 2)          |
| <code>cartToPolar()</code>   | core    | Conversion grad. cartésien → polaire         |
| <code>threshold()</code>     | imgproc | Seuillage binaire sur la magnitude           |
| <code>countNonZero()</code>  | core    | Comptage des pixels significatifs            |
| <code>meanStdDev()</code>    | core    | Statistiques sur les gradients de texture    |
| <code>blobFromImage()</code> | dnn     | Formatage de l'image pour YOLO               |
| <code>NMSBoxes()</code>      | dnn     | Non-Maximum Suppression                      |
| <code>rectangle()</code>     | imgproc | Dessin des boîtes englobantes                |
| <code>putText()</code>       | imgproc | Affichage des labels et scores               |
| <code>addWeighted()</code>   | core    | Fusion de deux images (superposition)        |

Table 3: Fonctions OpenCV utilisées dans ce TP

### 4. Architecture du pipeline

Le pipeline se décompose en deux branches parallèles qui se rejoignent à l'étape de validation :

- 1. Branche YOLO** (étapes 1–6) : configuration, chargement modèle, création blob, inférence, extraction détections, NMS.
- 2. Branche Sobel** (étape 7) : pour chaque boîte survivante, extraction ROI + validation par 3 méthodes Sobel + vote majoritaire.
- 3. Visualisation** (étapes 8–9) : codage couleur, labels, debug, sauvegarde et affichage.

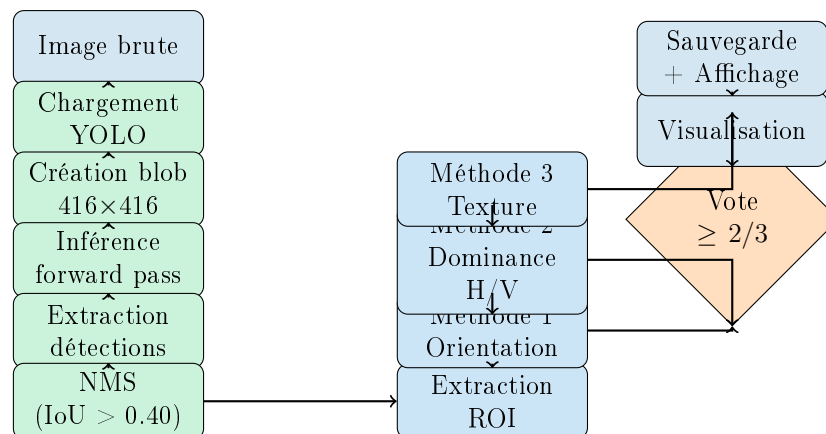


Figure 6: Architecture complète du pipeline de détection de piétons hybride

## 5. Étapes détaillées du pipeline

### 5.1 Étape 1 : Configuration et chargement de l'image

**Principe :** Lecture des arguments ligne de commande, chargement de l'image et vérification de son intégrité.

**Rôle :** Point d'entrée du pipeline ; tout arrêt précoce en cas d'erreur évite des plantages en aval.

```

1  int main(int argc, char** argv) {
2      if (argc < 2) {
3          cout << "Usage: " << argv[0]
4              << " <input_image.jpg> [output_image.jpg]\n";
5          return -1;
6      }
7      string inputPath = argv[1];
8      string outputPath = (argc >= 3) ? argv[2] : "output_detected.
9          jpg";
10
11      Mat img = imread(inputPath);
12      if (img.empty()) {
13          cerr << "ERREUR: Impossible de lire l'image: " << inputPath
14              << endl;
15          return -1;
16      }
17      cout << "Taille: " << img.cols << "x" << img.rows << "\n";
18  }

```

Listing 1: Configuration et chargement de l'image

### 5.2 Étape 2 : Chargement du modèle YOLO

**Principe :** Chargement du réseau de neurones pré-entraîné à partir des fichiers de configuration (.cfg) et de poids (.weights).

**Rôle :** Préparer le modèle de deep learning pour l'inférence.

```

1 Net net = readNetFromDarknet(MODEL_CFG, MODEL_WEIGHTS);
2 if (net.empty()) {
3     cerr << "ERREUR: Modele YOLO non trouve\n";
4     return -1;
5 }
6 net.setPreferableBackend(DNN_BACKEND_OPENCV);
7 net.setPreferableTarget(DNN_TARGET_CPU);
8 vector<String> outNames = net.getUnconnectedOutLayersNames();

```

Listing 2: Initialisation du reseau YOLO

### Fichiers requis

- yolov4-tiny.cfg – architecture du réseau (Darknet)
- yolov4-tiny.weights – poids pré-entraînés sur COCO ( $\approx 23$  Mo)

YOLOv4-tiny produit deux couches de sortie (détection à deux échelles).

## 5.3 Étape 3 : Création du blob d'entrée

**Principe :** Transformation de l'image en un tenseur 4D normalisé compatible avec l'entrée de YOLO.

**Rôle :** Formater les données pour qu'elles respectent le format attendu par le réseau.

```

1 // 1/255.0 = normalisation, Size(416,416) = redimensionnement,
2 // true = swapRB (BGR -> RGB)
3 Mat blob = blobFromImage(img, 1/255.0,
4                           Size(INPUT_WIDTH, INPUT_HEIGHT),
5                           Scalar(0, 0, 0), true, false);
6 net.setInput(blob);

```

Listing 3: Transformation de l'image en blob

## 5.4 Étape 4 : Inférence YOLO (forward pass)

**Principe :** Passage du blob à travers toutes les couches du réseau pour obtenir les prédictions brutes.

**Rôle :** Exécution de la détection par deep learning ; étape la plus coûteuse en calcul.

```

1 vector<Mat> outputs;
2 net.forward(outputs, outNames); // outputs[0] et outputs[1] = 2
   echelles

```

Listing 4: Forward pass du reseau

## 5.5 Étape 5 : Extraction des détections brutes

**Principe :** Parcours des sorties du réseau pour extraire les boîtes, les scores de confiance et la classe, en ne conservant que les piétons (`classId == 0`) au-dessus du seuil de confiance.

**Rôle :** Convertir les sorties brutes du réseau en détections interprétables.

```

1  for (size_t i = 0; i < outputs.size(); ++i) {
2      float* data = (float*)outputs[i].data;
3      for (int row = 0; row < outputs[i].rows;
4          ++row, data += outputs[i].cols) {
5          Mat scores = outputs[i].row(row).colRange(5, outputs[i].
6              cols);
7          Point classIdPoint;
8          double conf;
9          minMaxLoc(scores, nullptr, &conf, nullptr, &classIdPoint);
10
11         if (conf > CONF_THRESHOLD && classIdPoint.x == 0) {
12             float centerX = data[0] * img.cols;
13             float centerY = data[1] * img.rows;
14             float w        = data[2] * img.cols;
15             float h        = data[3] * img.rows;
16
17             int left      = max(0, (int)(centerX - w/2));
18             int top       = max(0, (int)(centerY - h/2));
19             int width     = min((int)w, img.cols - left);
20             int height    = min((int)h, img.rows - top);
21
22             boxes.emplace_back(left, top, width, height);
23             confidences.push_back(static_cast<float>(conf));
24         }
25     }

```

Listing 5: Extraction et filtrage des détections YOLO

## 5.6 Étape 6 : Non-Maximum Suppression (NMS)

**Principe :** Suppression des boîtes redondantes en ne gardant que celle avec le score le plus élevé parmi toutes les boîtes dont l'IoU mutuel dépasse le seuil NMS.

**Rôle :** Éviter les multiples détections du même piéton.

```

1  vector<int> indices;
2  NMSBoxes(boxes, confidences,
3      CONF_THRESHOLD,    // seuil de confiance minimum : 0.35
4      NMS_THRESHOLD,    // seuil IoU pour suppression : 0.40
5      indices);

```

Listing 6: Application de la NMS

## 5.7 Étape 7 : Validation Sobel (3 méthodes + vote)

Pour chaque boîte survivante au NMS, une ROI (*Region of Interest*) est extraite et soumise aux trois méthodes de validation :

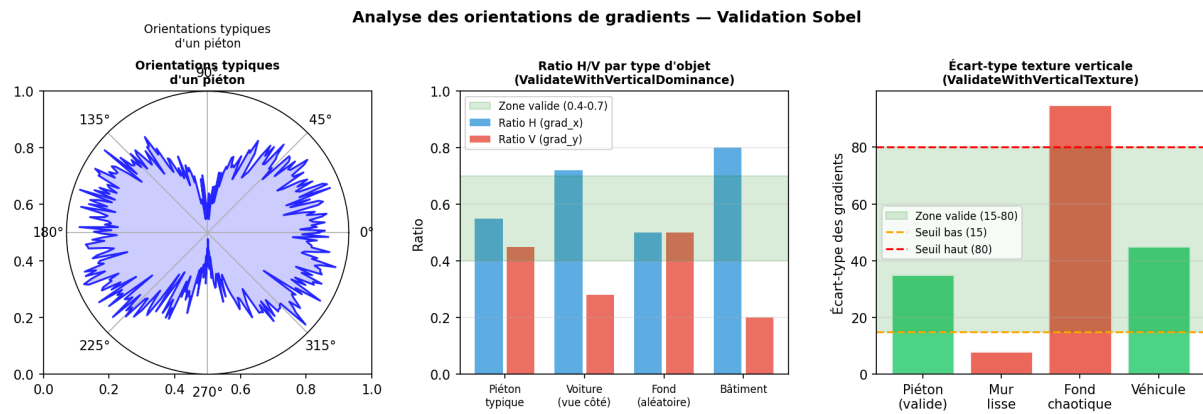


Figure 7: Les 3 méthodes de validation Sobel : orientations (gauche), ratio H/V (centre), écart-type texture (droite)

### Méthode 1 — Analyse des orientations (validateWithOrientationAnalysis)

**Principe :** Utilisation de `cartToPolar()` pour obtenir l'angle précis de chaque gradient. On compte les pixels significatifs (magnitude > 50) dont l'angle est < 20° ou > 160° (gradients quasi-horizontaux = contours verticaux).

```

1  Sobel(gray, grad_x, CV_32F, 1, 0, 3);
2  Sobel(gray, grad_y, CV_32F, 0, 1, 3);
3
4  Mat magnitude, angle;
5  cartToPolar(grad_x, grad_y, magnitude, angle, true); // true =
   degrees
6
7  threshold(magnitude, significant_mask,
8            SOBEL_MAGNITUDE_THRESHOLD, 255, THRESH_BINARY);
9
10 int vertical_count = 0;
11 for (int y = 0; y < angle.rows; y++) {
12     for (int x = 0; x < angle.cols; x++) {
13         if (significant_mask.at<uchar>(y, x) > 0) {
14             float ang = angle.at<float>(y, x);
15             if (ang < 20 || ang > 160) vertical_count++;
16         }
17     }
18 }
19 double vertical_ratio = (double)vertical_count / total_significant;
20 return vertical_ratio >= VERTICAL_RATIO_THRESHOLD; // >= 0.30

```

Listing 7: Analyse des orientations des gradients

### Méthode 2 — Dominance verticale (validateWithVerticalDominance)

**Principe :** Calcul du ratio entre la somme des gradients horizontaux (`abs_grad_x`) et le total des gradients. Un piéton présente un mélange équilibré : ni trop de gradients horizontaux (mur), ni trop de verticaux (sol).

```

1 Scalar sum_x = sum(abs_grad_x);
2 Scalar sum_y = sum(abs_grad_y);
3
4 double total_grad = sum_x[0] + sum_y[0];
5 if (total_grad < 1000) return false;
6
7 double horizontal_dominance = sum_x[0] / total_grad;
8 return (horizontal_dominance > 0.4 && horizontal_dominance < 0.7);

```

Listing 8: Calcul du ratio de dominance H/V

### Méthode 3 — Texture verticale (validateWithVerticalTexture)

**Principe :** Utilisation de la dérivée seconde verticale (Sobel d'ordre 2) pour mesurer la régularité périodique des structures : l'alternance jambes/fond ou bras/corps produit un écart-type dans l'intervalle [15, 80].

```

1 Sobel(gray, grad_y, CV_16S, 0, 2, 3);
2 convertScaleAbs(grad_y, grad_y);
3
4 Scalar meanVal, stddev;
5 meanStdDev(grad_y, meanVal, stddev);
6
7 return (stddev[0] > 15 && stddev[0] < 80);

```

Listing 9: Analyse de texture par dérivée seconde

### Vote majoritaire

**Principe :** Les trois méthodes votent indépendamment. La détection est confirmée si au moins 2 méthodes sur 3 renvoient `true`. Ce système de vote rend la validation robuste : une méthode peut échouer (ROI partielle, posture atypique) sans invalider l'ensemble.

```

1 bool hasStrongVerticalStructure(const Mat& roi) {
2     if (roi.empty() || roi.rows < 20 || roi.cols < 10) return false;
3
4     bool orientation_ok = validateWithOrientationAnalysis(roi);
5     bool dominance_ok   = validateWithVerticalDominance(roi);
6     bool texture_ok     = validateWithVerticalTexture(roi);
7
8     int votes = (orientation_ok ? 1 : 0)
9                 + (dominance_ok   ? 1 : 0)
10                + (texture_ok     ? 1 : 0);
11
12     return votes >= 2;
13 }

```

Listing 10: Fonction de validation globale par vote

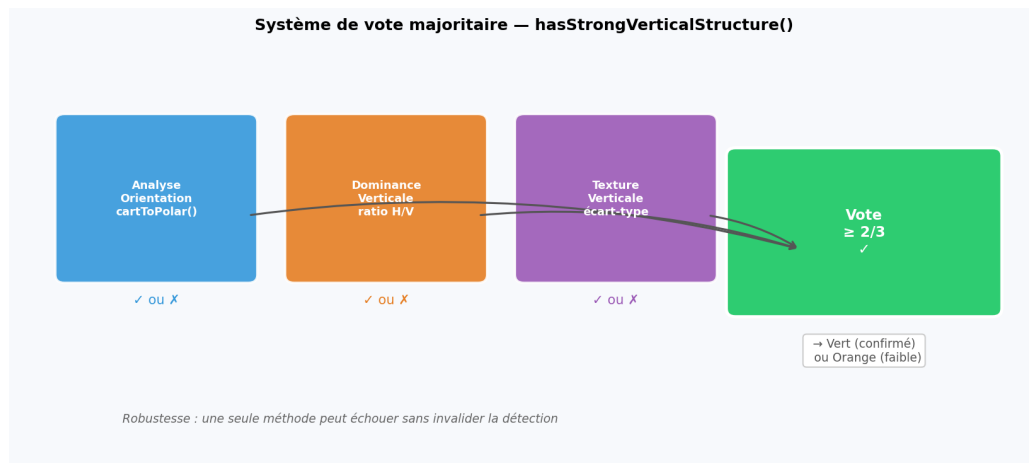


Figure 8: Système de vote majoritaire combinant les 3 méthodes Sobel

| Méthode             | Information extraite                      | Seuil de validation | Poids vote    |
|---------------------|-------------------------------------------|---------------------|---------------|
| Analyse orientation | % gradients $< 20^\circ$ ou $> 160^\circ$ | $\geq 30\%$         | 1             |
| Dominance H/V       | Ratio gradient horizontal / total         | [0.4, 0.7]          | 1             |
| Texture verticale   | Écart-type dérivée seconde $y$            | [15, 80]            | 1             |
| Décision finale     |                                           |                     | Vote $\geq 2$ |

Table 4: Récapitulatif des méthodes de validation Sobel

## 5.8 Étape 8 : Visualisation et codage couleur

**Principe :** Dessin des boîtes englobantes avec codage couleur selon le résultat de la validation Sobel, ajout des labels et génération d'une image de debug.

**Rôle :** Visualisation claire : **vert** = confirmé par Sobel, **orange** = YOLO seul sans confirmation.

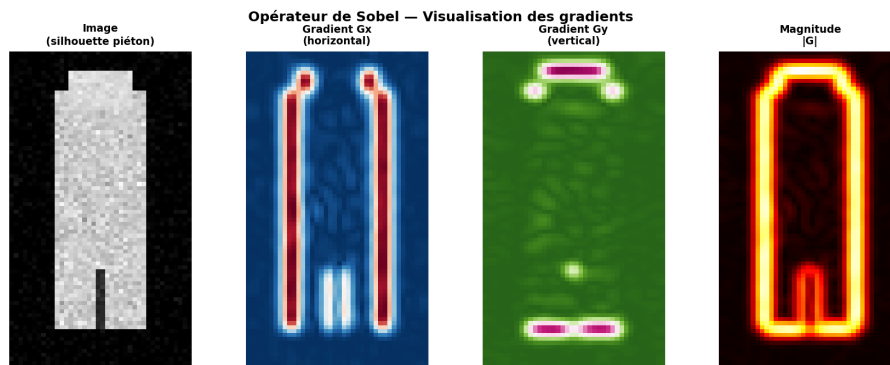
```

1 Scalar color = classical_ok ? Scalar(0, 255, 0)
2                               : Scalar(0, 165, 255);
3
4 rectangle(result, box, color, 3);
5
6 string label = "Personne " + to_string(i + 1) + " "
7               + to_string(int(conf * 100 + 0.5)) + "%";
8 string sobel_status = classical_ok ? "OK Sobel" : "Sobel faible";
9 label += " " + sobel_status;
10
11 Size textSize = getTextSize(label, FONT_HERSHEY_SIMPLEX,
12                               0.55, 2, &baseline);
13 Point textOrg(box.x, box.y - 8);
14 rectangle(result,
15             textOrg + Point(0, baseline),
16             textOrg + Point(textSize.width, -textSize.height),
17             color, FILLED);
18 putText(result, label, textOrg,

```

```
19 FONT_HERSHEY_SIMPLEX, 0.55, Scalar(255,255,255), 2);
```

Listing 11: Dessin des detections avec codage couleur

Figure 9: Image de debug Sobel : bleu = gradient horizontal ( $G_x$ ), rouge = gradient vertical ( $G_y$ )

### 5.9 Étape 9 : Sauvegarde et affichage final

**Principe :** Écriture de l'image résultat sur le disque et affichage dans trois fenêtres : résultat annoté, gradients Sobel de debug, image originale.

```
1  imwrite(outputPath, result);
2
3  namedWindow("Detection Piétons + Validation Sobel", WINDOW_NORMAL);
4  imshow("Detection Piétons + Validation Sobel", result);
5
6  if (indices.size() > 0) {
7      namedWindow("Visualisation Sobel", WINDOW_NORMAL);
8      imshow("Visualisation Sobel", debug_sobel);
9  }
10 namedWindow("Image originale", WINDOW_NORMAL);
11 imshow("Image originale", img);
12
13 waitKey(0);
14 destroyAllWindows();
```

Listing 12: Sauvegarde et affichage



## 6. Travail pratique à réaliser

---

### 6.1 Partie 1 : Prise en main

1. Installez OpenCV et téléchargez les fichiers YOLO (voir Annexe 8.1).
2. Compilez et exécutez le programme sur les images fournies.
3. Observez le codage couleur vert/orange : quels piétons passent la validation Sobel ?
4. Testez avec différentes images (piétons seuls, groupes, environnements variés).

### 6.2 Partie 2 : Analyse des paramètres

1. **Seuils YOLO** : modifiez `CONF_THRESHOLD` (0.2 à 0.8) et observez l'impact.
2. **Seuils Sobel** : ajustez `SOBEL_MAGNITUDE_THRESHOLD` et `VERTICAL_RATIO_THRESHOLD`.
3. **NMS** : testez différentes valeurs de `NMS_THRESHOLD` (0.3 à 0.6).
4. Quel paramètre a le plus d'influence sur la qualité de la détection ?

## 7. Questions théoriques

---

1. Pourquoi combine-t-on deep learning et vision classique plutôt que d'utiliser seulement YOLO ?
2. Expliquez la différence entre un gradient horizontal ( $G_x$ ) et un gradient vertical ( $G_y$ ). Que détectent-ils ?
3. Pourquoi un piéton présente-t-il un équilibre entre gradients horizontaux et verticaux ?
4. Quel est l'avantage d'utiliser `cartToPolar()` plutôt que de calculer manuellement l'orientation ?
5. Comment le seuil de magnitude `SOBEL_MAGNITUDE_THRESHOLD` influence-t-il la détection ?
6. Pourquoi utilise-t-on un vote majoritaire plutôt qu'une moyenne des scores ?
7. Quelles sont les limites de cette approche ? Dans quels cas échouera-t-elle ?

## 8. Annexes

### 8.1 Installation et compilation

```

1 # Installation d'OpenCV (Ubuntu/Debian)
2 sudo apt update && sudo apt install libopencv-dev
3
4 # Verification
5 pkg-config --modversion opencv4
6
7 # Telechargement des fichiers YOLO
8 wget https://github.com/AlexeyAB/darknet/raw/master/cfg/yolov4-tiny
   .cfg
9 wget https://github.com/AlexeyAB/darknet/releases/download/
   darknet_yolo_v4_pre/yolov4-tiny.weights

```

Listing 13: Installation OpenCV et telechargement YOLO

```

1 # Compilation standard
2 g++ -o detection_pietons detection_pietons.cpp \
3     'pkg-config --cflags --libs opencv4'
4
5 # Avec optimisations
6 g++ -O3 -march=native -o detection_pietons detection_pietons.cpp \
7     'pkg-config --cflags --libs opencv4'
8
9 # Execution
10 ./detection_pietons image_test.jpg resultat.jpg

```

Listing 14: Compilation et execution

### 8.2 Rappel — Opérateurs de convolution

| Opérateur | Noyau $3 \times 3$                                                        | Effet                                              |
|-----------|---------------------------------------------------------------------------|----------------------------------------------------|
| Sobel X   | $\begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$ | Contours verticaux ( $\partial I / \partial x$ )   |
| Sobel Y   | $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$ | Contours horizontaux ( $\partial I / \partial y$ ) |
| Laplacien | $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$   | Tous les contours ( $\nabla^2 I$ )                 |

Table 5: Opérateurs de convolution courants

### 8.3 Paramètres recommandés

| Paramètre                 | Valeur par défaut | Plage d'ajustement |
|---------------------------|-------------------|--------------------|
| CONF_THRESHOLD            | 0.35              | 0.20 – 0.60        |
| NMS_THRESHOLD             | 0.40              | 0.30 – 0.55        |
| SOBEL_MAGNITUDE_THRESHOLD | 50                | 30 – 80            |
| VERTICAL_RATIO_THRESHOLD  | 0.30              | 0.20 – 0.45        |

Table 6: Paramètres ajustables du système

### 8.4 Dépannage

#### Problèmes courants et solutions

- **Cannot read image** : vérifiez le chemin et les permissions du fichier.
- **Modèle YOLO non trouvé** : placez `.cfg` et `.weights` dans le même dossier.
- **Erreur de compilation** : vérifiez que `libopencv-dev` est installé.
- **Détections manquées** : baissez `CONF_THRESHOLD` ou `SOBEL_MAGNITUDE_THRESHOLD`.
- **Trop de faux positifs** : augmentez `CONF_THRESHOLD` et/ou renforcez les seuils Sobel.

### 8.5 Références

1. Redmon, J., & Farhadi, A. (2018). *YOLOv3: An Incremental Improvement*.
2. Bochkovskiy, A. et al. (2020). *YOLOv4: Optimal Speed and Accuracy of Object Detection*.
3. Sobel, I., & Feldman, G. (1968). *A 3x3 isotropic gradient operator for image processing*.
4. Documentation OpenCV : <https://docs.opencv.org/>